

AGENTIC AI · AUTONOMOUS LOGISTICS · URBAN AIR QUALITY

EcoShield Delhi

Agentic AI Pollution-Control System

A decentralised fleet of anti-smog water-sprinkler tankers, routed across the Delhi Metro network by a central AI coordinator running a modified multi-objective Dijkstra solver.

[krishiv47](#)

krishiv47.github.io/ecoshield-delhi-ai

github.com/krishiv47/ecoshield-delhi-ai

THE PROBLEM

Delhi's air is a moving, time-critical target

PM2.5 hotspots flare unpredictably with traffic surges, industrial haze, inversions and **road & construction dust resuspended by passing traffic**. Static smog towers cannot follow the pollution — mitigation has to be mobile, predictive and coordinated.

250+

PM2.5 (ug/m3) marks a "severe" hotspot needing intervention

6+

Major junctions flare at once during peak events

minutes

The window to act before a spike worsens

Goal: mitigate hotspots **before** sensors confirm them, coordinate many trucks as one fleet, and keep every unit fuelled, watered and optimally routed — autonomously.

THE SOLUTION

One brain, many mobile actuators

Central AI Coordinator

Global telemetry of the network and fleet. Predicts spikes, dispatches sub-fleets, handles exceptions, enforces the supply lifecycle.

Mobile tanker fleet

Heterogeneous water-sprinkler trucks (3000 / 5000 / 12000 L) with IoT misting cannons. Drivers steer; the AI dictates the route.

Delhi Metro network

~50 real stations across all lines plus 6 water/charging depots form the directed road graph the fleet operates on.

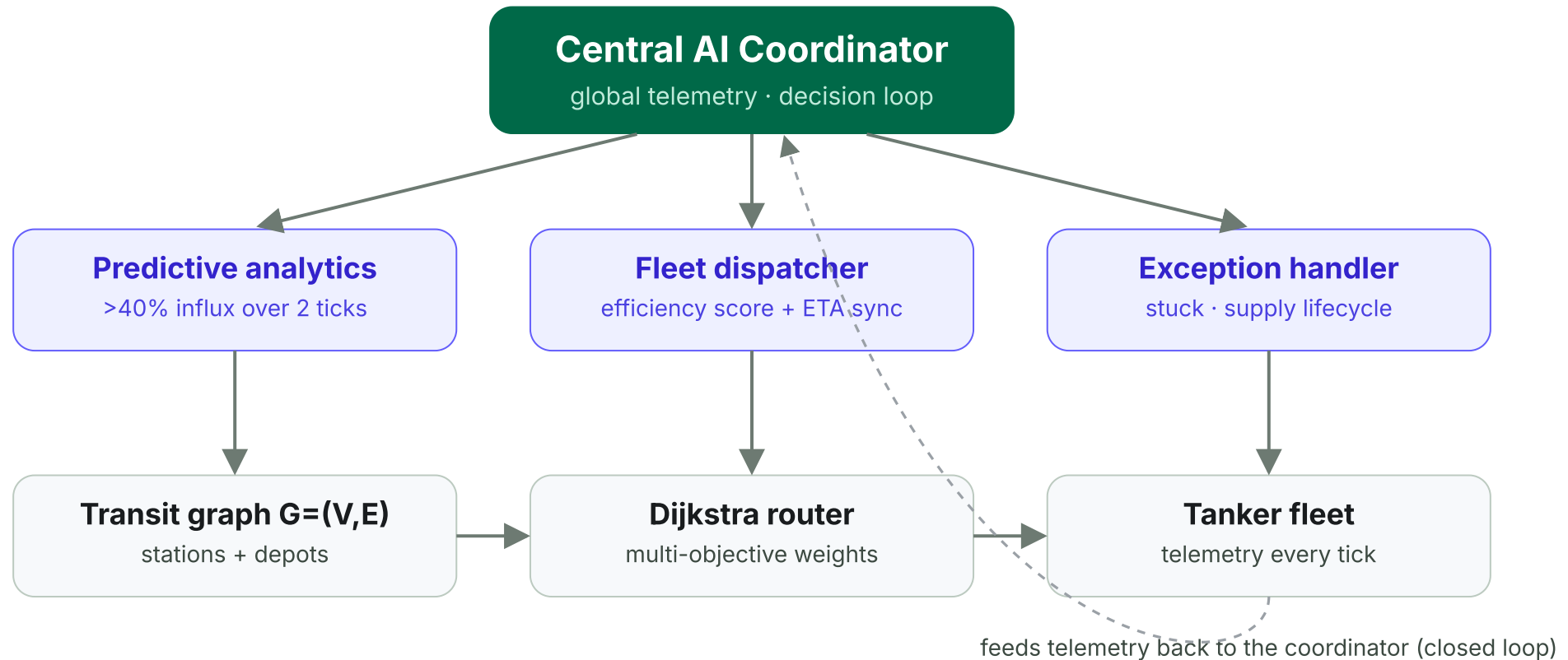
Road cleaning & dust suppression

A dedicated fleet of **10 road-cleaning trucks** continuously patrols and **wet-washes the road surface** — binding settled dust so it **stays on the ground** instead of being kicked back up as PM10 / PM2.5 by passing traffic.

Why "agentic"

Perceive (live AQI + traffic), reason (predict, score, route), act (dispatch, spray, replenish) in a continuous closed loop — with no human in the routing decision.

Strict OOD, clean dependency flow



DOMAIN MODEL

A directed graph with live, stateful telemetry

- **Nodes (V):** metro stations carrying dynamic **PM2.5** and **traffic influx**, with rolling history for trend analytics.
- **Special nodes:** Sewage Treatment Plants (recycled water) and CNG/EV depots (energy) — modelled as service subclasses via an abstract base.
- **Edges (E):** stateful road segments whose congestion factor and emission penalty mutate every tick.
- **Geometry:** every edge snapped to real road shapes (OSRM) so paths follow actual streets.

50

metro stations, all lines

6

water / charging depots

134

directed weighted edges

26

trucks: 16 sprinklers + 10
road cleaners

Truck state machine: **IDLE** **EN_ROUTE** **SPRAYING** **STUCK** **REPLENISHING**

Modified multi-objective Dijkstra

Edge weight is not static distance — it blends congestion-stretched travel time with a dynamic, vehicle-aware emission overhead.

```
# cost of crossing edge e with a given vehicle
W(e) = ( distance_km / base_speed ) × traffic_density_factor + emission_penalty

# levied only when a heavy chassis idles into an already-critical cell
emission_penalty = COEF × emission_weight(vehicle) × traffic_factor × severity
```

Vehicle-aware

Mini / Medium / Heavy get different optimal paths over the same graph at the same instant.

Dynamic

Weights recomputed from live telemetry each query; gridlock reroutes onto open bypasses.

Efficient

Binary-heap single-source Dijkstra, $O((V+E) \log V)$, run in Python and in the browser.

Four autonomous behaviours, every tick

1 Predictive spike pre-emption

A >40% influx rise over two ticks flags an imminent breach and sends a pre-emptive curtain before the sensor confirms it.

2 Workload split + ETA sync

An efficiency-score matrix picks a 2-3 truck sub-fleet and staggers departures so each arrives as the previous empties.

3 Stuck-in-traffic exceptions

Below 5 km/h: mist in place if inside the zone, or revoke the route and re-allocate the load to an unblocked unit.

4 Closed-loop supply lifecycle

Below 15% water / 20% fuel, the AI locks the console, routes to the nearest depot, then reintegrates at 95%.

The two "stuck in traffic" branches

Inside the hotspot zone

Stationary misting curtain

If a truck is trapped within the target zone, it engages its cannons in place — a pitch-adjusted curtain knocking down the idling exhaust of the surrounding gridlock. The mission is kept.

Far from the hotspot

En-route interception

If trapped mid-transit, the AI revokes the target, locks the console, marks the unit STUCK, recomputes the graph, and re-allocates the workload to a moving unit with an open bypass.

Recovery: stuck units rejoin the reserve automatically once their segment clears; the displaced workload is topped up by the next dispatch pass.

A live, data-driven admin dashboard

Built on a Material-style design system (Tailwind + Inter), rendered over a real Leaflet map of Delhi. Five working views, one continuously-running simulation.

Admin Control

Live map, hotspots, fleet, event stream

Fleet Ops

Roster of every tanker; open a driver console

Metro Network

Station air quality across all lines

Routing Engine

Interactive Dijkstra path finder

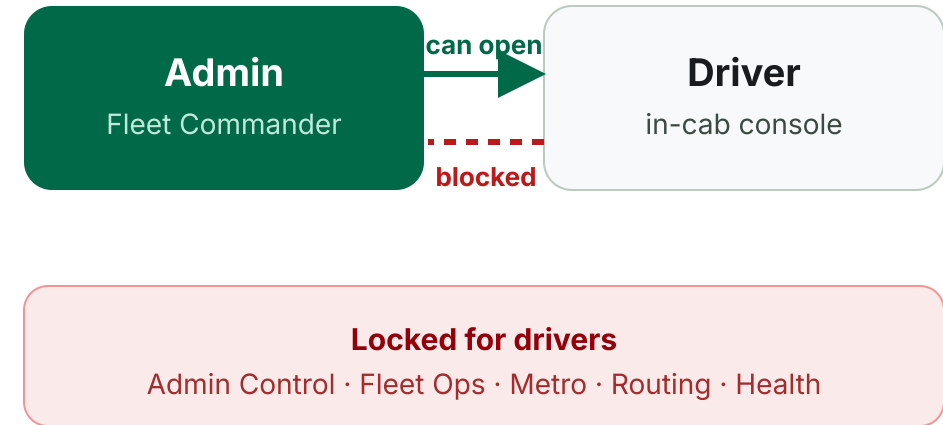
System Health

Platform & coordinator telemetry

Tankers shown **filled = cleaning**, **hollow = resting in reserve**; metro lines in their official colours; gridlock highlighted live.

The human steers, the AI dictates

- A restricted in-cab **Driver Console** shows the assigned mission, the Dijkstra route, and water/charge gauges.
- Admin modules are **locked** for drivers — bounded actuator only.
- The admin can open **any** driver's console (one-way hierarchy).
- A "console locked" banner appears whenever the AI overrides the route.



Production-grade, fully reproducible

Core engine

Python 3.10+, standard library only. Strict OOD: dataclasses, enums, ABCs, type hints, binary-heap Dijkstra.

Front end

Tailwind + Inter design system, Leaflet map on OpenStreetMap/CARTO. No build step; Mappls-ready via one key.

Delivery

Public GitHub repo, GitHub Pages deploy, continuously-running build, 28 passing unit tests.

28

unit + integration tests passing

0

third-party runtime dependencies

Live

krishiv47.github.io/ecoshield-delhi-ai

What it demonstrates — and what's next

Demonstrated

- Continuous multi-agent coordination over a real metro network
- Predictive, split, exception and supply workflows firing live
- Hotspots mitigated tick-over-tick while a reserve still rests

Roadmap

- Live IoT AQI + traffic feeds replacing the simulated environment
- Mappls / MapmyIndia basemap and turn-by-turn directions
- GTFS metro alignments and reinforcement-learned dispatch

The architecture is data-source agnostic — swapping the simulated feed for real telemetry needs no change to the agents or the solver.

THANK YOU

An autonomous shield for Delhi's air

Perceive, predict, dispatch, mitigate — continuously, and without a human in the routing loop.

Live demo · krishiv47.github.io/ecoshield-delhi-ai

Source · github.com/krishiv47/ecoshield-delhi-ai